

Langage C

TD/TP n° 5: Les structures

Dr KENMOGNE EDITH BELISE

Questions de Cours : Schématisation d'une structure, désignation d'un champ de structure, et désignation d'un champ d'une structure pointée

On considère le bout de code suivant.

```
struct ville {
    char nom[30];
    int nbre //nombre d'habitants
    float s; // superficie en km²
}
void main() {
    struct ville v1;
    struct ville v2={"Dschang", 225, 120000};
    struct ville tab_ville1[5];
    struct ville tab_ville2[5]= {{ "Dschang", 225, 120000}, {"Douala", 923, 2800000}};
    struct ville *pv;
    struct ville *tab_pv[2];
    v1.nom[0]="Kribi";
    v1.nbre=350000;
    v1.s=891;
    tab_pv[0]=&v1;
    tab_pv[1]=&v2;
    pv=&v2;
    return;
}
```

- 1) Schématiser l'évolution des variables après l'exécution de chaque instruction.
- 2) Donner les valeurs de *pv->nom* et de *v2.nom* après l'exécution du bout de code.
- 3) Après l'exécution de ce bout de code, **pv* et *v2* désignent-ils la même chose ?
- 4) Après l'exécution de ce bout de code, *pv->nom*, *(*pv).nom* et *v2.nom* désignent-ils la même chose ?
- 5) Donner des expressions équivalentes à *pv->nom*.
- 6) Donner des expressions équivalentes à *v2.nom*.

Questions de Cours : Définition de nouveau type avec typedef

Rappel :

Le mot réservé *typedef* permet de définir de nouveaux type en langage C. Le bout de code qui suit illustre l'utilisation de *typedef* par la définition de quatre nouveaux types : *String*, *Ptr*, *VILLE* et *Tab_ville*.

```
typedef char String[255]; //Definition du nouveau type String. Un objet String est un vecteur de 255 char.
typedef int *Ptr; // Definition du nouveau type Ptr. Un objet Ptr est un pointeur sur int.
typedef struct ville {
    char nom[30];
    int nbre //nombre d'habitants
    float s; // superficie en km²
} VILLE; // Definition du nouveau type VILLE. Un objet VILLE est un objet struct ville.
typedef VILLE Tab_ville[250]; // Un objetTab_ville est un vecteur de 255 VILLE.
```

Le bout de code qui suit déclare des objets des quatre types définis ci-dessus.

```
String s, tab[10]; // Déclaration d'un String s et d'un tableau de dix Strings nommé tab.
Ptr ptr; // Déclaration d'un Ptr nommé ptr.
Ptr f(int); //Déclaration du prototype de la fonction f. Elle retourne un objet Ptr.
VILLE v1, v2, v3; // Déclaration de trois objets VILLE.
```

Tab_ville tv1, tv2, tv3 ; //Déclaration de trois objets de Tab_ville (qui sont des tableaux de villes).

Questions :

- 1) Redéfinir le type VILLE en supposant qu'on a déjà défini « *struct ville* » comme à l'exercice 1.
- 2) Donner une déclaration équivalente à la quatrième déclaration de variables (*VILLE* v1, v2, v3 ;).
- 3) Pour chacune des quatre autres déclarations de variable, donner une déclaration équivalente.

Problème 1 : Analyse de code

<pre>#define SIZE 10 struct rectangle{ float long ; //longueur float larg ; //largeur }; typedef struct rectangle RECTANGLE; typedef RECTANGLE Tab_rect[SIZE] ; struct rectangle* lire1() { rectangle *pr ; float v ; pr=(rectangle*) malloc(sizeof(rectangle)) ; printf(" entrer la longueur) ; scanf("%f", &v) ; pr->long=v ; printf(" entrer la largeur) ; scanf("%f", &v) ; pr->larg=v ; return pr ; } void lire2(struct rectangle *pr) { printf(" entrer la longueur) ; scanf("%f", &(pr->long)) ; printf(" entrer la largeur) ; scanf("%f", &((*pr).larg)) ; return ; }</pre>	<pre>float perimetre(struct rectangle r){ return (r.long + r.larg)*2 ; } float superficie1(struct rectangle r){ return r.long*r.larg ; } void afficher(struct rectangle r) { printf("Rectangle[%f,%f]", r.long, r.larg) ; } float superficie2(Tab_rect t, int taille){ int i ; float s=0 ; for (i=0 ; i<taille ; i++) s=s+superficie(t[i]) ; return s ; } float maximum(Tab_rect t, int taille){ int i ; float max ; max=superficie(t[0]) ; for (i=1 ; i<taille ; i++) { s=superficie(t[i]) ; if (s>max) max=s ; } return max ; }</pre>
--	---

- 1) Dire ce que fait chaque fonction
- 2) Pour chaque fonction définie ci-dessus, donner un entête (ou prototype) equivalent
- 3) Ecrire une fonction *minimum* qui prend en paramètre un tableau de rectangles et le nombre d'éléments qu'il contient puis retourne la position contenant la superficie minimale. Retourner -1 s'il est vide.
- 4) Ecrire une fonction qui prend en paramètre un entier *n*, crée un tableau d'une capacité de *n* rectangles, lit les dimensions de *n* rectangles, stocke ces dimensions dans le tableau crée et retourne ledit tableau.
- 5) Ecrire une fonction qui trie un tableau de rectangles par superficie croissante. On pourrait utiliser le technique du trie par insertion décrite sur la fiche de TD sur les fonctions.

Problème 2 : Gestion des commandes

Un produit est caractérisé par son code, son libellé (une chaîne de caractères) et le prix unitaire. Une commande simple est constituée d'un code produit et la quantité commandée. Une commande est un ensemble de commandes simples.

Partie I : Structures de Données

- 1) Définir un type *PRODUIT*. Illustrer cette définition par un schéma.
- 2) Définir un type *COM_S* (commande simple) constitué de deux champs : le code du produit et la quantité. Illustrer cette définition par un schéma.
- 3) Définir un type *COMMANDE* constitué de quatre champs : Le premier champ est le code de la commande. Le deuxième champ est un tableau de commandes simples. Le troisième champ est la taille du tableau, le quatrième champ est le nombre de commandes simples stockées dans le deuxième champ. Le quatrième champ représente aussi la prochaine case libre du tableau. Il faudrait l'appeler *libre*. Illustrer la définition de *COMMANDE* par un schéma.
- 4) Définir un type *LISTE_PRODUIT* constitué de trois champs : Le premier champ est un tableau de produits. Le deuxième champ est la taille du tableau. Le troisième champ est le nombre de produits stockés dans le premier champ. Le troisième champ représente aussi la prochaine case libre du tableau. Il faudrait l'appeler *libre*. Illustrer la définition de *LISTE_PRODUIT* par un schéma.

Partie II : Opérations sur un produit

Définir les fonctions suivantes.

- 1) *PRODUIT* lire_produit1()* lit les caractéristiques d'un produit au clavier et retourne l'adresse d'une zone mémoire contenant les valeurs lues.
- 2) *void lire_produit2(PRODUIT *p)* lit les caractéristiques d'un produit au clavier et les stocke dans **p*.
- 3) *void affiche_prod1(PRODUIT *p)* affiche le produit **p* à l'écran.
- 4) *void affiche_prod2(PRODUIT pro)* affiche le produit *pro* à l'écran.

Partie III : Opérations sur une liste de produits

- 1) *LISTE_PRODUIT * creation_lp(int n)* crée une liste de produit pouvant contenir *n* produits. Le premier champ de l'objet crée est une zone mémoire obtenue dynamiquement et pouvant contenir *n* produits.
- 2) *int vide_lp(LISTE_PRODUIT lp)* retourne TRUE si le tableau de la liste de produits *lp* est vide (ne contient pas de produit) et FALSE sinon.
- 3) *int plein_lp(LISTE_PRODUIT lp)* retourne TRUE si le tableau de la liste de produits *lp* est plein et FALSE sinon.
- 4) *int ajout_lp(LISTE_PRODUIT *plp, PRODUIT p)* copie le produit *p* dans la première case libre du premier champ de la liste de produits **plp* et retourne TRUE (pour dire que l'ajout a réussi). La fonction doit retourner FALSE si le tableau de **plp* est plein.
- 5) *int search_lp1(LISTE_PRODUIT lp, char *code)* recherche le produit dont le code du produit associé est *code* et retourne la position dudit produit s'il existe dans *lp* et -1 sinon.
- 6) *PRODUIT* search_lp2(LISTE_PRODUIT lp, char *code)* recherche le produit dont le code du produit associé est *code* et retourne un pointeur sur ledit produit s'il existe dans *lp* et NULL sinon.
- 7) *int delete_lp(LISTE_PRODUIT *plp, char *code)* recherche le produit dont le code est *code*, supprime ledit produit de **plp* et retourne TRUE (pour dire que la suppression a réussi) et retourne FALSE si le produit recherché n'existe pas dans **plp*.

Partie IV: Opérations sur une commande simple

- 1) *COM_S* lire_cs1()* lit les caractéristiques d'une commande simple au clavier et retourne l'adresse d'une zone mémoire contenant les valeurs lues.
- 2) *void lire_cs2(COM_S *pcs)* lit les caractéristiques d'une commande simple au clavier et les stocke dans **pcs*.
- 3) *void affiche_cs1(COM_S cs)* affiche la commande simple *cs* à l'écran de manière à faire apparaître le code du produit et la quantité commandée.
- 4) *void affiche_cs2(COM_S cs, LISTE_PRODUIT lp)* affiche la commande simple *cs* à l'écran de manière à faire apparaître tous les champs du produit associé et la quantité commandée. On suppose que la liste *lp* contient tous les produits.
- 5) *float montant (COM_S cs, LISTE_PRODUIT lp)* retourne le montant de la commande simple *cs*.

Partie V: Opérations sur une commande

Définir les fonctions suivantes.

- 1) *COMMANDE* creation_cm(int n)* crée une commande pouvant contenir *n* commandes simples. Le

deuxième champ de la commande créée est une zone mémoire obtenue dynamiquement et pouvant contenir n commandes simples.

2) *int vide_cm(COMMANDE c)* retourne TRUE si le tableau de la commande c est vide (ne contient pas de commande simple) et FALSE sinon.

3) *int plein_cm(COMMANDE c)* retourne TRUE si le tableau de la commande c est plein et FALSE sinon.

4) *int ajout_cm(COMMANDE *pc, COM_S cs)* copie la commande simple cs dans la première case libre du premier champ de la commande *pc et retourne TRUE (pour dire que l'ajout a réussi). La fonction doit retourner FALSE si le tableau de *pc est plein.

5) *float montant (COMMANDE c,LISTE_PRODUIT lp)* retourne le montant de la commande c.

6) *int search_cm(COMMANDE c, char *code)* recherche la commande simple dont le code du produit associé est code et retourne la position de la dite commande simple si elle existe dans c et -1 sinon.

7) *int delete_cm(COMMANDE *pc, char *code)* recherche la commande simple dont le code du produit associé est code, supprime ladite commande simple de *pc et retourne TRUE (pour dire que la suppression a réussi) et retourne FALSE si la commande simple recherchée n'existe pas dans *pc.

8) *int maximum(COMMANDE c,LISTE_PRODUIT lp)* retourne la position de la commande simple dont le montant est maximal. La fonction doit retourner -1 si le tableau de c est vide.

9) *int minimum(COMMANDE c,LISTE_PRODUIT lp)* retourne la position de la commande simple dont le montant est minimal. La fonction doit retourner -1 si le tableau de c est vide.

10) *void affiche_cm1(COMMANDE c)* affiche le code d'une commande et toutes ses commandes simples. On utilisera *affiche_cs1*.

11) *void affiche_cm2(COMMANDE c, LISTE_PRODUIT lp)* affiche le code d'une commande et toutes ses commandes simples. *affiche_cs2*.